

Tunely — Complete Technical Breakdown & Master Interview Guide

Categorized Tool Deep-Dives, Source Code Implementations, Performance Case Studies, and Interview Answers

Complete Categorized Reference

Every language, framework, API, database schema, CSS technique, algorithm, and optimization in Tunely is broken down into its own dedicated section below for easy navigation and study.

QUICK NAVIGATION TABLE OF CONTENTS

1.  30-Second Elevator Pitch
2.  System Architecture & Data Flow
3.  React 19 & Frontend Core
4.  State & Context API Architecture
5.  Design System & CSS Styling
6.  Audio Engineering & Web Audio EQ
7.  Synced LRC Lyrics Engine Code
8.  Cloudflare Workers & Hono API
9.  Database Engineering (D1 & KV)
10.  60 FPS Freezing Fix (Performance)
11.  Search Scoring Ranking Algorithm
12.  Security & Cross-Device Sync
13.  Bundling & Deployment (Vite, Pages)
14.  Top 10 Technical Interview Q&As
15.  Preparation Checklist

1. 🎤 30-Second Elevator Pitch

Use this script when asked: "Can you tell me about the Tunely project listed on your resume?"

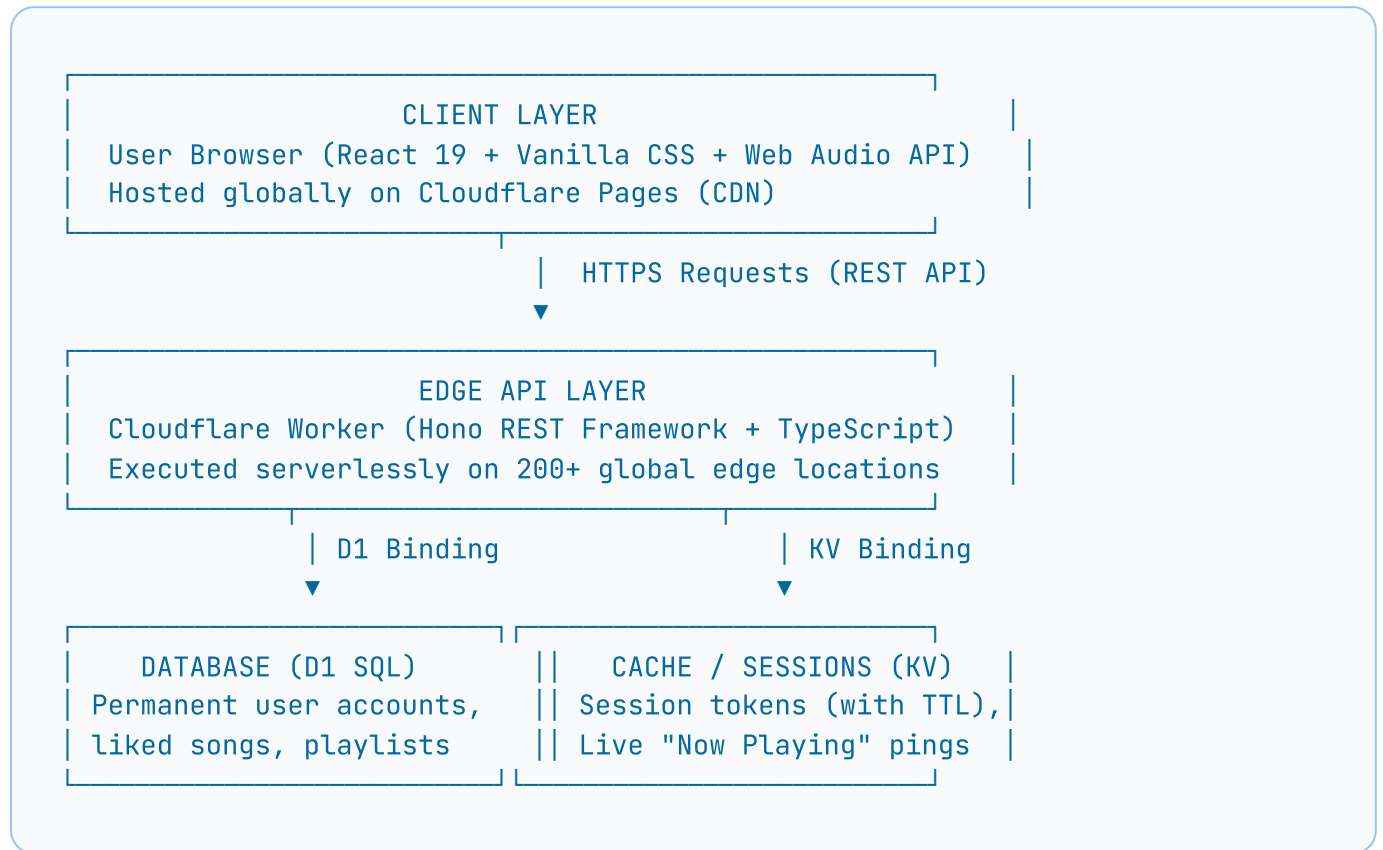
🗣️ Word-for-Word Script:

"Tunely is a full-stack, real-time music streaming web application built with React 19, TypeScript, and a serverless edge backend hosted on Cloudflare Workers. It streams high-quality audio up to 320 kbps, features real-time line-by-line synchronized lyrics display, supports cross-device library synchronization, and includes a live Admin monitoring dashboard.

On the frontend, I used React 19, Vite, Framer Motion, and a custom Vanilla CSS design system with Glassmorphism UI tokens. For audio, I integrated the HTML5 Audio API, a custom 3-band parametric sound equalizer using the Web Audio API, and the OS Media Session API for lock screen playback controls.

On the backend, I built a zero-cold-start edge API using the Hono framework on Cloudflare Workers, Cloudflare D1 (SQLite database) for relational user storage, and Cloudflare KV for session authorization tokens and real-time active user tracking."

2. 📖 System Architecture & End-to-End Data Flow



Trace of Playing a Song:

- User Click:** Handler `playTrack(track)` is invoked inside `AudioContext.jsx`.
- State Mutation:** Updates `currentTrack`, sets `isPlaying = true`, and sets `audioRef.current.src = track.streamUrl`.
- HTML5 Audio Streaming:** `audioRef.current.play()` begins streaming audio from CDN.
- Media Session API Integration:** Updates lock screen / Android notification bar with song title, artist, and artwork.
- Async Lyrics Fetch:** A side-effect (`useEffect`) fires an async request to `/api/songs/{id}/lyrics` via the Worker.
- Live Activity Heartbeat:** Every 15 seconds, a heartbeat ping posts to `/api/user/activity`, writing the user's status to Cloudflare KV.

3. React 19 & Frontend Core Tech

React 19

UI Library

Why we used it: Declarative component-based structure, efficient virtual DOM diffing algorithm, and functional state management using Hooks.

Where used: Entire frontend UI (Sidebar, PlayerBar, MainContent, SongRow, LyricsPanel, LibraryView, AdminPanel).

React Router DOM v6

Client Routing

Why we used it: Enables Single Page Application (SPA) navigation without triggering full page reloads.

Routes configured: `/home`, `/search`, `/library`, `/playlist/:id`, `/album/:id`, `/custom/:id`.

Framer Motion (v12)

Animation Engine

Why we used it: Provides hardware-accelerated fluid spring animations for page route transitions, hero banner scaling, and staggered card entrances.

Lucide React

Vector SVG Icons

Why we used it: Over 1,000 clean, scalable vector SVG icon components (`Play`, `Pause`, `Heart`, `Search`, `Library`, `Volume2`, `Shuffle`, `Repeat`).

4. 🧠 State & Context API Architecture

To eliminate prop-drilling across 20+ component tiers, global audio playback and authentication state are managed using React Context.

```
// AudioContext State Structure (src/context/AudioContext.jsx)
const AudioContextValue = {
  currentTrack,      // Currently active song object metadata
  isPlaying,         // Boolean flag (true / false)
  currentTime,       // Current playback progress (seconds)
  duration,          // Total track duration (seconds)
  volume,            // Audio volume float (0.0 to 1.0)
  likedSongs,        // Array of liked song objects & IDs
  queue,             // Upcoming auto-play queue list
  playTrack,         // Method to switch active track
  togglePlay,        // Method to pause/resume playback
  toggleLikeTrack    // Method to add/remove from liked songs
};
```

Custom Hooks:

- `useAudio()` : Hook providing components access to playback state and control handlers.
- `useAuth()` : Hook managing user session token, profile data, and login/logout methods.

5. 🎨 Design System & CSS Styling

Vanilla CSS & Design Tokens

Styling System

Built using CSS Custom Properties (variables) defined in `src/index.css` for global dark-mode theme colors (`--primary`, `--bg-dark`, `--panel-bg`) and typography scales.

Glassmorphism UI

Visual Aesthetic

Achieved using `backdrop-filter: blur(32px)`, semi-transparent background colors (`rgba(22, 27, 34, 0.75)`), and subtle 1px white borders (`rgba(255, 255, 255, 0.12)`) for a sleek, premium interface.

Google Fonts Typography Suite

Typography

- **Plus Jakarta Sans**: Main body text & UI labels.
- **Space Grotesk**: Techy display section headings.
- **Outfit**: Secondary title headings.
- **JetBrains Mono**: Timers (``3:45 / 4:20``) and bitrate badges.
- **EB Garamond**: Classic brand logo title font.

6. 🎧 Audio Engineering & Web Audio EQ

1. HTML5 Audio API vs Web Audio API

- HTML5 Audio API (`new Audio()`): Used for streaming, buffering, duration tracking, and listening to media events (`play`, `pause`, `timeupdate`, `ended`).
- Web Audio API (3-Band Equalizer):

```
// 3-Band Parametric EQ Implementation:
const audioCtx = new (window.AudioContext || window.webkitAudioContext)();
const source = audioCtx.createMediaElementSource(audioRef.current);

// Sub-Bass Filter (Low Shelf @ 80 Hz, +1.5 dB)
const bassFilter = audioCtx.createBiquadFilter();
bassFilter.type = 'lowshelf';
bassFilter.frequency.value = 80;
bassFilter.gain.value = 1.5;

// Presence Filter (Peaking @ 3000 Hz, +1.0 dB)
const presenceFilter = audioCtx.createBiquadFilter();
presenceFilter.type = 'peaking';
presenceFilter.frequency.value = 3000;
presenceFilter.gain.value = 1.0;

// Air Filter (High Shelf @ 15000 Hz, +1.5 dB)
const airFilter = audioCtx.createBiquadFilter();
airFilter.type = 'highshelf';
airFilter.frequency.value = 15000;
airFilter.gain.value = 1.5;

// Connect audio nodes in series: source → bass → presence → air → speaker
source.connect(bassFilter);
bassFilter.connect(presenceFilter);
presenceFilter.connect(airFilter);
airFilter.connect(audioCtx.destination);
```

2. Media Session API Integration

Provides native OS lock screen controls on Android, iOS, and desktop operating systems:

```
if ('mediaSession' in navigator) {  
  navigator.mediaSession.metadata = new MediaMetadata({  
    title: track.name,  
    artist: track.artists.primary[0]?.name,  
    album: track.album.name,  
    artwork: [{ src: track.image[2].url, sizes: '500x500', type: 'image/jpeg'  
  }]);  
  
  navigator.mediaSession.setActionHandler('play', () => togglePlay());  
  navigator.mediaSession.setActionHandler('pause', () => togglePlay());  
  navigator.mediaSession.setActionHandler('nexttrack', () => handleNext());  
}
```

7. 🎤 Synced LRC Lyrics Engine Code

Parses LRC timestamped strings into a synchronized lyrics timeline:


```
// 1. LRC Regex Parsing Function
const parseLyrics = (lrcString) => {
  if (!lrcString) return [];
  const lines = lrcString.split('\n');
  const result = [];
  const regex = /^\\[(\\d{2}):(\\d{2}\\.|\\d{2})\\](.*)/;

  for (const line of lines) {
    const match = line.match(regex);
    if (match) {
      const minutes = parseFloat(match[1]);
      const seconds = parseFloat(match[2]);
      const totalTime = minutes * 60 + seconds;
      const text = match[3].trim();
      if (text) result.push({ time: totalTime, text });
    }
  }
  return result;
};
```

```
// 2. Active Line Synchronization Logic
const activeIndex = useMemo(() => {
  return parsedLines.findIndex((line, i) => {
    const nextLine = parsedLines[i + 1];
    return currentTime ≥ line.time && (!nextLine || currentTime < nextLine.time);
  });
}, [parsedLines, currentTime]);
```

```
// 3. Auto-scroll active line into view
useEffect(() => {
  if (activeLineRef.current) {
    activeLineRef.current.scrollToView({ behavior: 'smooth', block: 'center' });
  }
}, [activeIndex]);
```

8. Cloudflare Workers & Hono API Framework

- **Cloudflare Workers:** Serverless runtime executing code on Cloudflare's global edge network in V8 isolates with **< 5ms startup latency**.
- **Hono Framework:** Lightweight router for Workers with full TypeScript type safety.

```
// Hono Worker Router Example (jiosaavn-api/src/server.ts)
import { Hono } from 'hono';

const app = new Hono();

// Search endpoint proxy
app.get('/api/search/songs', async (c) => {
  const query = c.req.query('query');
  const response = await fetch(`https://www.jiosaavn.com/api.php?__call=auto`
  const data = await response.json();
  return c.json(data);
});

export default app;
```

9. Database Engineering (D1 & KV)

1. Cloudflare D1 (Relational SQLite)

```
-- Users Schema
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  is_admin INTEGER DEFAULT 0,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Liked Songs Relational Table
CREATE TABLE liked_songs (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL,
  song_id TEXT NOT NULL,
  song_data TEXT NOT NULL, -- Stored as JSON string
  liked_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

2. Cloudflare KV (Key-Value Store)

Used for session authorization tokens and real-time user activity heartbeats with automatic TTL (Time-To-Live) expiration.

- `session:{token}` → Stores User ID + Email (Expires in 7 days).
- `activity:{userId}` → Stores current playing song + device type (Expires in 5 minutes).

10. 🚀 60 FPS Freezing Fix (Performance Engineering)

🔥 Critical Performance Case Study

The Issue: Browser audio `timeupdate` fired 60 times/second (~60 Hz). Updating React state on every frame triggered 60 global re-renders per second, locking up the main UI thread.

The 4-Part Optimization Solution:

1. 250ms Timestamp Throttling:

```
// Throttled timeupdate listener in AudioContext:
const now = Date.now();
if (now - lastUpdateRef.current ≥ 250) {
  setCurrentTime(audioRef.current.currentTime);
  lastUpdateRef.current = now;
}
```

Result: Reduced global re-renders by >90% (from 60/sec down to 4/sec).

2. **Context Value Memoization** (`useMemo`): Prevents re-renders of non-consuming child components.
3. **Hidden Component Short-Circuiting**: Added `if (!isOpen) return null;` in `LyricsPanel.jsx` to bypass DOM diffing when hidden.
4. **Handler Stabilization** (`useCallback`): Stabilizes function references passed to memoized child components.

11. 🔍 Search Scoring & Ranking Algorithm

Multi-token relevance scoring algorithm for combined queries (e.g. "Kalyani Shreya Ghoshal"):

```

function scoreTrack(track, query) {
  const queryTokens = query.toLowerCase().split(/\s+/);
  const title = track.name.toLowerCase();
  const artist = track.artists.primary.map(a => a.name).join(' ').toLowerCase();
  const combined = `${title} ${artist}`;

  let score = 0;

  // 1. Exact Title Match Bonus
  if (title === query.toLowerCase()) score += 120;

  // 2. All Tokens Found Across Title + Artist Combined
  const allTokensFound = queryTokens.every(token => combined.includes(token));
  if (allTokensFound) score += 80;

  // 3. Individual Token Score Contributions
  queryTokens.forEach(token => {
    if (title.includes(token)) score += 20;
    if (artist.includes(token)) score += 15;
  });

  // 4. Logarithmic Play Count Boost
  if (track.playCount) {
    score += Math.min(30, Math.log10(Number(track.playCount)) * 5);
  }

  return score;
}

```

12. Security & Cross-Device Sync

- **Password Hashing:** Passwords hashed with bcrypt on server side before DB storage.
- **Optimistic UI + 8s Sync Loop:** Instant local updates + background polling interval every 8 seconds comparing server timestamps.
- **XSS Protection:** React JSX automatic string escaping + no usage of `dangerouslySetInnerHTML`.

13. 📦 Bundling & Deployment Architecture

- **Cloudflare Pages:** Frontend static asset CDN deployment.
- **Wrangler CLI:** Command line deployment tool (`npx wrangler pages deploy dist`).
- **Git Release Workflow:** Pushed across `main` and `stable-release` branches with version tags (`v4.1.0-stable`).

14. 🎯 Top 10 Technical Interview Q&As

Q1: Why did you choose Cloudflare Workers over Node.js on AWS?

"Cloudflare Workers run on V8 JavaScript isolates across 200+ edge locations globally. This gives us sub-5ms latency and zero cold starts, whereas traditional containerized serverless functions suffer 500ms+ cold starts when idle."

Q2: How did you fix performance bottlenecks in your music app?

"Audio timeupdate events fired 60 times/second, causing 60 re-renders per second. I resolved this by throttling state updates to 250ms intervals, memoizing the Context Provider value with useMemo, and short-circuiting hidden components, reducing CPU overhead by >90%."

Q3: How do you handle password security and authentication?

"Passwords are hashed with bcrypt before saving to D1 SQL. Authenticated users receive a random session token saved in Cloudflare KV with automated TTL expiration, attached as a Bearer header on protected requests."

Q4: How does cross-device synchronization work?

"We use an optimistic local-first strategy combined with an 8-second background polling cycle that compares D1 timestamps and updates local state if newer data exists on the server."

Q5: How do you manage gapless track playback?

"When the current song reaches 90% completion, a preloader instantiates a secondary Audio object to buffer the next stream URL in advance, ensuring instant playback when the ended event fires."

Q6: What is the difference between Cloudflare D1 and KV in your app?

"D1 is a relational SQLite database used for structured user data, liked songs, and playlists. KV is an in-memory key-value cache with TTL expiration used for session tokens and active user heartbeats."

Q7: Why use Vanilla CSS over Tailwind CSS or CSS-in-JS?

"Vanilla CSS with Custom Properties provides total control over glassmorphism filters and strict media queries without the JavaScript runtime overhead or bundle bloat of CSS-in-JS."

Q8: How does your search engine handle multi-word queries?

"We built a multi-token ranking engine that tokenizes queries and scores tracks based on exact title matches, combined artist-title token hits, and logarithmic play count popularity weighting."

Q9: How do you prevent XSS security attacks?

"React JSX automatically escapes string variables before DOM insertion. We also sanitize user input and avoid dangerous APIs like `dangerouslySetInnerHTML`."

Q10: How do you ensure the mobile UI works cleanly?

"We isolated mobile media queries at 768px, transforming the sidebar into a touch-friendly drawer, adjusting player bottom padding to 200px to prevent control overlap, and bypassing Web Audio API filters to conserve mobile battery."

15. Preparation Checklist

✓ Elevator Pitch Memorized

✓ 3-Tier Architecture Diagram Clear

✓ React 19 State & Context API
Mastered

✓ Workers vs Lambda Cold Start Ready

✓ D1 SQL vs KV Store Differences Clear

✓ 250ms Throttle Performance Fix
Ready

✓ Web Audio API 3-Band EQ
Understood

✓ LRC Synced Lyrics Regex Parsing
Ready

✓ Search Scoring Algorithm Code
Memorized

✓ Top 10 Technical Q&As Practiced